

课程笔记

REINFORCE 算法在语言模型中的应用

基于公开课程资料整理

2025

极简/直观 REINFORCE 4 LLM

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 20 分钟

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言	2
2 Reward 环境设计	2
2.1 基于 Cosine Similarity 的 Reward	2
3 REINFORCE 实现	2
3.1 算法流程	2
3.2 本章小结	2
4 总结与延伸	2

1 引言

本期将 REINFORCE 算法应用到语言模型中，实现一个简单效果：不管输入什么 prompt，模型输出都包含特定语义（如 “mode”）。

2 Reward 环境设计

2.1 基于 Cosine Similarity 的 Reward

Reward 定义

1. 语言模型 rollout 生成一个序列（句子）
2. 计算该句子的 embedding
3. 计算目标词（如 “mode”）的 embedding
4. 两者的 cosine similarity 作为 reward

3 REINFORCE 实现

3.1 算法流程

1. 从当前策略 π_θ 采样生成序列
2. 计算每个序列的 reward（cosine similarity）
3. 计算 PG Loss 并反向传播更新 θ

3.2 本章小结

REINFORCE 是最朴素的 PG 算法，可以直接应用于语言模型，将 RL 的概念落地到实际的文本生成任务中。

4 工程提示：reward 设计要服务于训练反馈

这节课虽然重点在 reward 环境设计与 REINFORCE 实现，但真正落地时最关键的问题是：reward 是否能稳定地区分“值得强化”和“不值得强化”的行为。如果 reward 太稀疏、太晚出现，或者和最终目标错位，再正确的策略梯度实现也会学得很慢。

常见环境设计误区

- 只有最终成功奖励，没有中间反馈，导致 credit assignment 太难
- 奖励过于密集，却没有约束正确方向，导致模型钻空子
- 环境状态不稳定，让同一动作在不同轮次得到冲突反馈

4.1 本章小结

环境和 reward 设计决定了 RL 信号的质量。与其一味调算法，不如先确认奖励是否真的在推动目标行为出现。

5 总结与延伸

1. REINFORCE 可以直接用于语言模型的 RL 训练
2. Reward 环境可以用 embedding similarity 定义
3. 这是理解更复杂的 PPO/GRPO 的基础